

Global Solution 2026 - Cybersecurity

Módulo DevSecOps aplicado ao projeto SpaceClimate

Curso	Engenharia de Software
Disciplina	Cybersecurity
Professor	MSc. Oerton Fernandes
Tema integrador	Segurança Cibernética em Soluções Baseadas no Ecossistema Espacial
Repositório	https://github.com/mathhfarias/gs-cybersecurity-spaceclimate

1. Contexto do Projeto

O SpaceClimate é uma API modelo baseada no ecossistema espacial. A solução simula o uso de dados de telemetria, clima e alertas ambientais, representando um sistema capaz de consumir informações orbitais e apoiar decisões relacionadas a riscos climáticos.

Como esse tipo de aplicação pode utilizar APIs externas, tokens de acesso, infraestrutura em nuvem e dados sensíveis, foi criado um módulo DevSecOps para incluir segurança no ciclo de desenvolvimento. O objetivo é reduzir riscos no pipeline e impedir que falhas críticas ou segredos expostos sejam integrados à branch principal.

Integrantes

Nome	RM
MATHEUS FARIAS DE LIMA	RM554254
MIGUEL MAURICIO PARRADO PATARROYO	RM554007
VITOR PINHEIRO NASCIMENTO	RM553693
GABRIEL LEÃO	RM552642
PEDRO HENRIQUE NARDACI CHAVES	RM553988

Entregáveis contemplados: mapeamento de riscos, controles de segurança, diagrama de pipeline, implementação prática, evidências, simulação e relação com ODS.

2. Mapeamento de Riscos no Pipeline

A tabela apresenta os principais riscos identificados no ciclo de desenvolvimento e entrega da API SpaceClimate.

Risco	Impacto	Tema relacionado	Controle sugerido
Vazamento de tokens, chaves de API ou arquivo .env	Acesso indevido a APIs externas, dados climáticos ou serviços em nuvem.	Gestão de Segredos	Uso de .env.example, GitHub Secrets e bloqueio de arquivos sensíveis.
Dependências vulneráveis	Exploração de falhas em bibliotecas usadas pela API.	Ferramentas de Segurança CI/CD	Scan automatizado com Trivy no GitHub Actions.
Configurações inseguras no Dockerfile	Execução de contêiner com padrões inseguros.	Segurança em Contêineres / Segurança como Código	Correção para usuário não-root e HEALTHCHECK.
Permissões excessivas no pipeline	Workflow com acesso maior do que o necessário.	Zero Trust	Permissões mínimas no workflow.
Falta de auditoria do processo de entrega	Falhas podem passar sem registro.	Monitoria e Auditoria Contínua	Logs, artefatos e histórico no GitHub Actions.

Os riscos se concentram em segredos, dependências, configurações, permissões e rastreabilidade. Por isso, os controles foram posicionados no pipeline antes da integração com a branch principal.

3. Controles de Segurança Escolhidos

3.1 Análise de Vulnerabilidades com Trivy

O Trivy foi integrado ao GitHub Actions para verificar vulnerabilidades, segredos e configurações inseguras. O relatório é publicado como artefato, permitindo auditoria posterior.

3.2 Gestão de Segredos

O projeto usa .env.example apenas como modelo. Arquivos reais como .env, .env.local e .env.production não devem ser versionados. O script scripts/check-secrets.sh bloqueia o pipeline quando um arquivo .env é encontrado.

3.3 Zero Trust aplicado ao pipeline

Nenhuma alteração é considerada confiável automaticamente. O código precisa passar por verificação de segredos, testes e análise de segurança antes do merge. O workflow também usa permissões reduzidas.

4. Diagrama do Pipeline DevSecOps

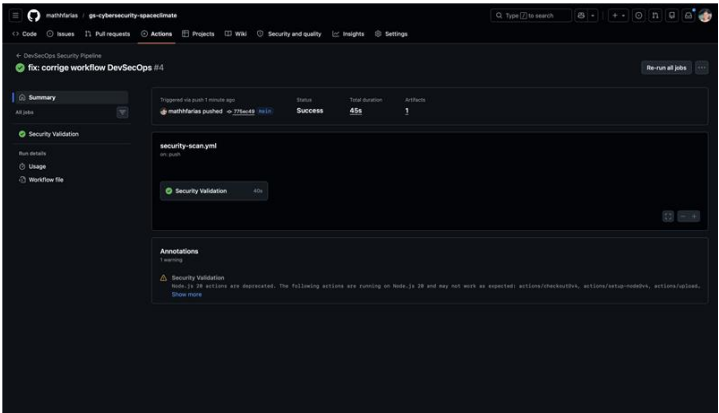


O pipeline atua como uma barreira de segurança: alterações com risco são detectadas antes de entrarem na branch principal.

5. Implementação Prática

A implementação prática foi uma checagem automatizada no GitHub Actions. O workflow principal fica em `.github/workflows/security-scan.yml` e executa verificação de segredos, instalação das dependências, testes automatizados e análise com Trivy.

O relatório do Trivy mostrou 0 vulnerabilidades no `package-lock.json` e apontou 2 melhorias no `Dockerfile`: uso de usuário não-root e adição de `HEALTHCHECK`. Essas melhorias foram tratadas na sequência como ação corretiva.



Evidência 1 - Pipeline executado com sucesso após ajuste do workflow.

Report Summary

Target	Type	Vulnerabilities	Secrets	Misconfigurations
package-lock.json	npm	0	-	-
Dockerfile	dockerfile	-	-	2

Legend:
- '?': Not scanned
- 'V': Clean (no security findings detected)

Dockerfile (dockerfile)
=====

Tests: 27 (SUCCESSES: 25, FAILURES: 2)
Failures: 2 (LOW: 1, MEDIUM: 0, HIGH: 1, CRITICAL: 0)

DS-0002 (HIGH): Specify at least 1 USER command in Dockerfile with non-root user as argument
Running containers with 'root' user can lead to a container escape situation. It is a best practice to run contain
See <https://avd.aquasec.com/misconfig/ds-0002>

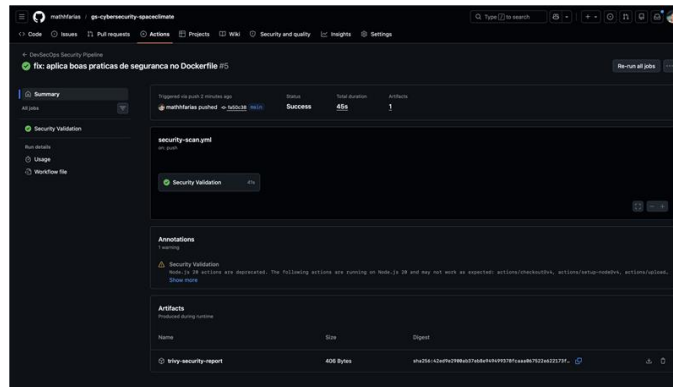
DS-0026 (LOW): Add HEALTHCHECK instruction in your Dockerfile
You should add HEALTHCHECK instruction in your docker container images to perform the health check on n
See <https://avd.aquasec.com/misconfig/ds-0026>

Evidência 2 - Relatório do Trivy com resumo da análise e melhorias apontadas no Dockerfile.

6. Ação Corretiva e Mini Simulação

6.1 Correção do Dockerfile

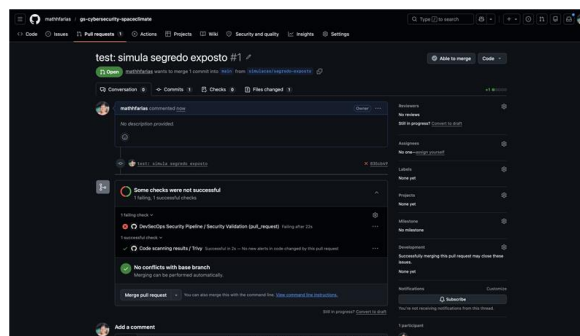
Após a auditoria automatizada, o Dockerfile foi corrigido com execução por usuário não-root e HEALTHCHECK. O commit de correção gerou uma nova execução bem-sucedida no pipeline, mantendo o artefato trivy-security-report.



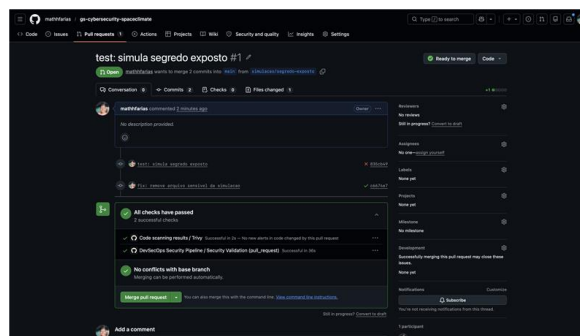
Evidência 3 - Pipeline aprovado após aplicar boas práticas de segurança no Dockerfile.

6.2 Simulação de segredo exposto

Foi criada uma branch de simulação com um arquivo .env falso. O controle scripts/check-secrets.sh detectou o arquivo sensível e o GitHub Actions bloqueou o pipeline. Depois, o arquivo foi removido e o Pull Request passou novamente.



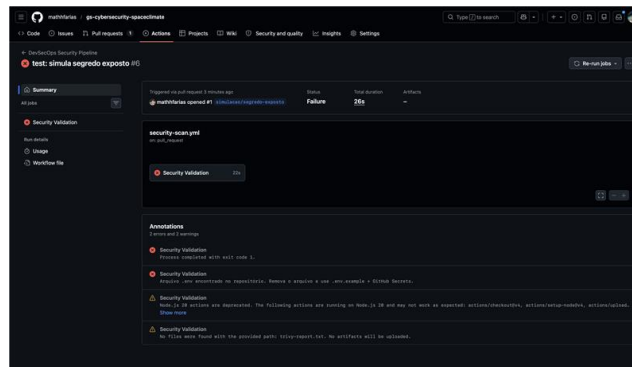
Evidência 4 - Pull Request com falha controlada por segredo exposto.



Evidência 5 - Após remoção do .env, os checks passaram e o PR ficou apto para merge.

7. Resultado da Simulação

A simulação demonstrou o ciclo completo: criação do problema, detecção automática, bloqueio do pipeline, correção e liberação do merge. O print da execução com falha mostra a mensagem: arquivo `.env` encontrado no repositório, removendo a alteração e usando `.env.example` ou GitHub Secrets.



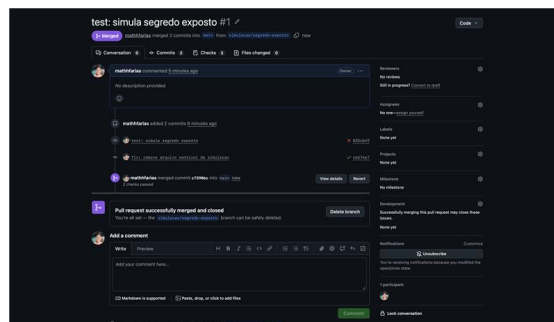
Evidência 6 - Detalhe da falha do GitHub Actions causada pelo arquivo `.env`.

8. Conexão com os ODS

A solução se relaciona com o ODS 9 - Indústria, Inovação e Infraestrutura, por demonstrar automação e segurança no ciclo de entrega; com o ODS 11 - Cidades e Comunidades Sustentáveis, por simular alertas climáticos; e com o ODS 13 - Ação Contra a Mudança Global do Clima, por usar o contexto de dados espaciais e climáticos.

9. Conclusão

O projeto SpaceClimate integrou práticas de DevSecOps de forma objetiva e verificável. O repositório apresenta API modelo, testes, pipeline YAML, script de bloqueio de segredos, Trivy, evidências, simulação e ação corretiva. Com isso, atende aos requisitos de mapeamento de riscos, controles de segurança, implementação prática, mini simulação e organização da entrega.



Evidência 7 - Pull Request da simulação corrigida, aprovado e mesclado.